```
import os
import sys
import subprocess
import urllib.request
import torch
import torchvision.datasets as dsets
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, TensorDataset
from torchvision.utils import save_image
from torchmetrics.image.inception import InceptionScore

REPO = "stylegan2-ada-pytorch"
REPO_URL = "https://github.com/NVlabs/stylegan2-ada-pytorch.git"
MODEL_URL = "https://nvlabs-fi-cdn.nvidia.com/stylegan2-ada-pytorch/pretrained/ffhq.pkl"
MODEL_PATH = "ffhq.pkl"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


def clone_repo():
    if not os.path.exists(REPO):
        subprocess.run(["git", "clone", REPO_URL], check=True)
    sys.path.insert(0, os.path.abspath(REPO))


def download_model():
    if not os.path.exists(MODEL_PATH):
        urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
```

```
def generate_images(num_images=10):
    import legacy

    with open(MODEL_PATH, "rb") as f:
        G = legacy.load_network_pkl(f)["G_ema"].to(device)

    os.makedirs("generated_images", exist_ok=True)
    images = []

    for i in range(num_images):
        z = torch.randn([1, G.z_dim], device=device)
        label = torch.zeros([1, G.c_dim], device=device)

        with torch.no_grad():
            img = G(
                z,
                label,
                truncation_psi=0.5,
                noise_mode="const"
            )
            img = ((img + 1) / 2).clamp(0, 1)

        save_image(img, f"generated_images/fake_{i}.png")
        images.append(img.cpu())

    return torch.cat(images)
```

```
def calculate_inception_score(imgs):
    imgs = (imgs * 255).clamp(0, 255).to(torch.uint8)

    loader = DataLoader(
        TensorDataset(imgs),
        batch_size=10
    )

    metric = InceptionScore()

    for batch in loader:
        metric.update(batch[0])

    mean, std = metric.compute()
    print(f"Inception Score: {mean:.2f} +/- {std:.2f}")



def prepare_real_images(num_images=10):
    dataset = dsets.CelebA(
        root="./data",
        split="train",
        transform=transforms.Compose([
            transforms.Resize(512),
            transforms.CenterCrop(512),
            transforms.ToTensor()
        ]),
        download=True
    )
```

```
    loader = DataLoader(dataset, batch_size=1, shuffle=True)
    os.makedirs("real_images", exist_ok=True)

    for i, (img, _) in enumerate(loader):
        if i >= num_images:
            break
        save_image(img, f"real_images/real_{i}.png")



def calculate_fid():
    subprocess.run([
        sys.executable,
        "-m",
        "pytorch_fid",
        "real_images",
        "generated_images"
    ], check=True)



def main():
    clone_repo()
    download_model()
    imgs = generate_images()
    calculate_inception_score(imgs)
    prepare_real_images()
    calculate_fid()
```

```
if __name__ == "__main__":
    main()
```